Niagara 4 point deployment · Part 2 of 2

Collecting Those Points

The station as a client: devices, points, proxy extensions, proof

Verified on Niagara 4.14.0.162 · every reading driven from outside the station

Part 1 covers the other direction: publishing station points as BACnet, oBIX, Modbus slave

Shanghai Gline Net Co., Ltd.

@jason912 · jason.zhang@gline-net.com

Four steps, and the third one carries the binding

Network device point with a proxy extension confirmation

Step	Object to create	What it establishes
1 · Network	driver network of the client flavour, enabled	the transport every device object below it uses
2 · Device	one object per field controller (address, port, poll)	the endpoint each point is addressed against
3 · Point	point with a proxy extension bound to one address	a value slot that actually polls
4 · Confirm	value read back over the network	a measurement in place of a status

The proxy extension is the binding

A point is a value slot, a status slot and a name. The proxy extension is the part that reaches the far end, and it is a protected slot: it is created with the point and cannot be attached afterwards. Pass it at creation and the point polls; leave it out and the object is a value holder with nothing behind it.

> cp from a point that already carries the right extension is the working shortcut, but it copies the extension's settings too dataType, dataAddress and byteOrder come along and are read back on every copied point.

Building the chain over the API

Four calls per protocol, and the one field that decides whether the point polls

Step	Operation	What it establishes
1 · Network	add + enabled = true	a live transport for every object under it
2 · Device	add, then ipAddress / port / deviceAddress	the far endpoint, address by address
3 · Point	add with PROXY = <module>:<ProxyExt>	the extension that reaches the register
4 · Proxy slots	oBIX PUT for dataType and dataAddress	which register, decoded how
5 · Read-back	two scheduler reads, then the external client	collection, confirmed twice

The point call in full

```
{"OP":"add","PATH":"/Drivers/ModbusTcpNetwork/<device>/points",
 "N":"<point>","T":"control:NumericPoint",
 "PROXY":"modbusCore:ModbusClientNumericProxyExt"}
```

proxyExt is a protected slot: it is created with the point and is neither added afterwards nor given a child. PROXY at creation is the route.

cp from a point that already carries the right extension also works and brings dataType, dataAddress and byteOrder with it.

> The same four calls serve all four protocols; only the module, the object types and the address slots change. The next page is the per-protocol column of that table.

The client chain per protocol

Same four steps, four different object names

	Modbus TCP	BACnet/IP	SNMP	oBIX
Network	modbusTcp:ModbusTcpNetwork	bacnet:BacnetNetwork	nSnmp:SnmpNetwork	obix:ObixNetwork
Device	ModbusTcpDevice	BacnetDevice	SnmpDevice	oBIX device
Address	ipAddress, port,deviceAddress	address(BacnetAddress)	ipAddress, port,community	URL path
Proxy ext	ModbusClientNumericProxyExt	BacnetNumericProxyExt	SnmpNumericProxyExt	ObixNumericProxyExt
Polled by	devicePollConfig+ pollScheduler	pollFrequency,COV	pollScheduler	poll rate
Outside check	FC03 / FC06	ReadProperty	snmpget	oBIX GET

Two distinctions worth keeping

nSnmp:SnmpNetwork is the nDriver type; the legacy snmp:SnmpNetwork still exists and behaves differently. One SNMP network per station.

BACnet collects on COV as well as polling, so normalCount on a BACnet network is read together with the COV counters before the scheduler is judged.

> Pick the module first, then the device type, then the proxy extension: the three names come from the same module and mixing them produces a type that the station will not create.

What each reading confirms

Configure over the API, then measure the two are different questions

Reading	Where it comes from	What it confirms
network enabled = true	REST read after mod	the transport was accepted
device status {ok}	REST read of the device	the address was accepted by the driver
health = Ok [timestamp]	REST read, PingMonitor	the far end answers on its ping address
point out = 227.00 {ok}	REST read of the point	a value exists and its status is ok
totalPolls increasing	two reads, ten seconds apart	the scheduler is running
snmpget 227 on the same OID	a client off-station	the number is the far end's, not a default

The pair that settles it

```
point   /Drivers/SnmpNetwork1/<device>/points/<point>/out   227.00 {ok}
snmpget  -v2c -c <community> <HOST> <OID>                   227
```

the same number in two processes: one collected, one measured

Status slots describe configuration and reachability. The value is the only thing a far end can corroborate so every protocol in this part is closed with its own client reading the same quantity.

A point that carries a value and an ok status is confirmed by reading that value independently, not by re-reading the slot.

Two readings confirm the scheduler

Read the counter twice, then read the value from outside

pollScheduler is the queue firing?

/Drivers/<driver network>/pollScheduler

Reading	Where	What it showed
totalPolls	first read	22511
totalPolls	second read, 10 s later	22596 the queue is firing
normalCount	same component	current = 1, the range in the queue

The same counter, two networks, one station

Network	Scheduler class	totalPolls	normalCount
SnmpNetwork1	NPollScheduler	22k and climbing	current = 1

A counter that advances between two reads is proof the scheduler ran; the value read from outside is proof of what it collected. Both belong in the same confirmation.

devicePollConfig describes intent. The counter and the external value describe outcome read all three, then state only what the outcome shows.

Close the loop from outside the station

Configure nothing, trust nothing, drive the far end

1. **Read the far end independently**
   Modbus master, snmpget, BACnet ReadProperty from another host

2. **Write a distinctive value at the source**
   a number you would recognise, or a quantity that moves

3. **Watch the station point**
   it follows to the new value with {ok}

Worked loop: one slave, one master, one station

```
FC06 write HR[1] = 7777      12 34 00 00 00 06 01 06 00 01 1e 61
FC03 read  HR[0..1]          12 34 00 00 00 07 01 03 04 04 d2 1e 61
slave point Srv_Reg1         7777.0 {ok}   (register 0 = 1234, 1 = 7777)
```

Three statements agree: the frame on the wire, the two register words, and the point's own value. That is the level of evidence every row in the checklist carries.

The same loop applies in the other direction: a client point is confirmed by driving its far end and reading the point back one row per protocol in VERIFIED.md.

Order of confirmation

Five readings, each one narrowing where a number can come from

#	Read	What it confirms
1	driver network enabled and {ok}	the transport is up, so device objects can poll
2	pollScheduler.totalPolls, twice	the queue behind those points is firing
3	devicePollConfig for the device	the range and dataType the point was registered with
4	device health timestamp	the far end answers on its ping address
5	the point's value against a client off-station	the number came from the far end

Two combinations that read the same and do not mean the same

Health advancing and a value that matches the far end: collection is confirmed end to end, and that is the state every completed row in the checklist is in.

Health advancing with a value that cannot be corroborated from outside means the address and the ping address are different questions read the point's dataAddress next to the device's pingAddress.

> Steps 1 to 4 are reads only. The fifth is the one that produces evidence, and it is the only step that needs a second process.

Every claim has a row

The checklist marks what was reproduced, and leaves the rest blank

niagara-ai-lab/VERIFIED.md lists each capability with the channel that performed it and the observation that confirmed it. A tick means the capability was built over the API and reproduced from outside the station; a blank cell means no claim is made for it.

Part	What it covers	Confirmed rows
1.1 1.18	publishing: BACnet/IP server, oBIX exports, Modbus slave, REST v2 operations	all ticked
1.19 1.21	publishing rows still open: ReadProperty on an export, oBIX /set/, SNMP exports	left blank
2.1 2.8	collecting: client network, device, point with proxy extension, scheduler, SNMP	all ticked
2.9 2.11	collecting rows still open: Modbus FC03, BACnet ReadProperty, oBIX server	left blank

A blank row is not a defect in the station; it is a row this lab has not reproduced yet, and the guides make no statement about it.

The marking rule is written at the bottom of the checklist: a slot read-back for the API steps, a protocol frame or a second process for the external steps, and two counter readings for anything about polling.

> Part 2 closes where it started: the API builds the chain, and a client outside the station is what turns a configuration into a measurement.